

Day 3, Session 4: Advanced Counters

Concept Block (12 min)

What We've Already Built

Day 2: You built a simple **2-bit binary counter** (counts $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$)

Today: We're adding **advanced features** that make counters useful in real systems.

The Four Advanced Counter Features

Real-world counters need more than just "count up forever":

1. Up-Down Counting (Bidirectional)

- Count up: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \dots$
- Count down: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

2. Load (Preset Value)

- Set counter to a specific value
- Example: Load 100, then count from there

3. Enable Control

- Count only when enabled
- When disabled, hold current value

4. Modulo-N (Custom Limit)

- Count from 0 to N-1, then wrap
 - Example: BCD counter ($0 \rightarrow 9 \rightarrow 0$)
-

Why These Features Matter

Real-world examples:

1. Up-Down Counter:

- **Elevator floor tracker** (up/down buttons)
- **Undo/Redo buffer** (navigate history forward/backward)
- **Bidirectional motor control** (position tracking)

2. Counter with Load:

- **Timer with preset** (microwave: load 60 seconds, count down)
- **Watchdog timer** (load timeout value, count down to reset)
- **UART baud rate generator** (load divisor, count to generate clock)

3. Counter with Enable:

- **Event counter** (count only when event occurs)

- **Frequency divider** (enable every N cycles)
- **Instruction counter** (count only when CPU is executing)

4. Modulo-N Counter:

- **BCD counter** (0-9 for decimal display)
- **Day-of-week counter** (0-6, Sunday to Saturday)
- **Traffic light sequencer** (0-2, Red/Yellow/Green)
- **Clock divider** (divide-by-10, divide-by-60)

Design 1: Up-Down Counter

Specification

Inputs:

- clk, rst: Standard clock and reset
- up_down: 1 = count up, 0 = count down

Output:

- count[7:0]: 8-bit counter value

Behavior:

- up_down=1: count increments (0→1→2→...→255→0)
- up_down=0: count decrements (255→254→...→1→0→255)

Implementation

```
// up_down_counter.v - 8-bit Up-Down Counter
module up_down_counter (
    input wire      clk,          // Clock
    input wire      rst,          // Async reset
    input wire      up_down,      // 1=up, 0=down
    output reg [7:0] count        // Counter value
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 8'd0;          // Reset to 0
        else begin
            if (up_down)
                count <= count + 1'b1; // Count up
            else
                count <= count - 1'b1; // Count down
        end
    end
endmodule
```

Line-by-Line Explanation

Line 13: count <= count + 1'b1;

- Increment: Add 1 to current count
- When count = 8'd255, next value wraps to 8'd0 (8-bit overflow)

Line 15: `count <= count - 1'b1;`

- Decrement: Subtract 1 from current count
- When `count = 8'd0`, next value wraps to `8'd255` (8-bit underflow)

Key insight: Verilog automatically handles wraparound for fixed-width arithmetic!

Design 2: Counter with Load

Specification

Inputs:

- `clk`, `rst`: Standard
- `load`: When HIGH, load 'data' into counter
- `data[7:0]`: Value to load

Output:

- `count[7:0]`: Counter value

Priority:

`Reset > Load > Count`

Implementation

`// counter_with_load.v - 8-bit Counter with Parallel Load`

```
module counter_with_load (
    input wire      clk,
    input wire      rst,
    input wire      load,           // Load control
    input wire [7:0] data,         // Data to load
    output reg [7:0] count
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 8'd0;           // Priority 1: Reset
        else if (load)
            count <= data;           // Priority 2: Load
        else
            count <= count + 1'b1;    // Priority 3: Count
    end
endmodule
```

Priority Logic Breakdown

```
if (rst)           // Highest priority
    count <= 8'd0;
else if (load)     // Second priority
    count <= data;
else               // Default: count
    count <= count + 1'b1;
```

Example sequence:

Time	rst	load	data	count	Action
0	1	0	50	0	Reset
10	0	0	50	0	Count: 0→1
20	0	0	50	1	Count: 1→2
30	0	1	50	2	Load 50
40	0	0	50	50	Count: 50→51
50	0	0	50	51	Count: 51→52

Design 3: Counter with Enable

Specification

Inputs:

- clk, rst: Standard
- en: Enable (1=count, 0=hold)

Output:

- count[7:0]: Counter value

Behavior:

- en=1: Counter increments
- en=0: Counter holds current value

Implementation

```
// counter_with_enable.v - 8-bit Counter with Enable
module counter_with_enable (
    input wire      clk,
    input wire      rst,
    input wire      en,           // Enable control
    output reg [7:0] count
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 8'd0;
        else if (en)
            count <= count + 1'b1;    // Count when enabled
        else
            count <= count;           // Hold when disabled
    end

endmodule
```

This is identical to the D flip-flop with enable pattern you learned!

Design 4: Modulo-N Counter (BCD Example)

Specification

BCD Counter: Counts 0 → 1 → 2 → ... → 9 → 0 (decimal digits)

Inputs:

- clk, rst: Standard

Output:

- count[3:0]: 4-bit counter (0-9)

Behavior:

- Counts: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, ...
- Skips values 10-15 (invalid BCD)

Implementation

```
// bcd_counter.v - BCD Counter (0-9)
module bcd_counter (
    input wire      clk,
    input wire      rst,
    output reg [3:0] count      // 4-bit for 0-9
);

always @(posedge clk or posedge rst) begin
    if (rst)
        count <= 4'd0;
    else begin
        if (count == 4'd9)           // At maximum?
            count <= 4'd0;           // Wrap to 0
        else
            count <= count + 1'b1; // Increment
    end
end

endmodule
```

Line-by-Line: Modulo Logic

Line 12: `if (count == 4'd9)`

- Check if counter reached maximum (9)

Line 13: count <= 4'd0;

- Wrap back to 0 (modulo behavior)

Line 15: `count <= count + 1'b1;`

- Normal increment when not at max

Sequence:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 0 \rightarrow 1 \rightarrow \dots$

↑
Wraps here

Parameterized Modulo-N Counter

Generalized version:

```
// modulo_n_counter.v - Parameterized Modulo-N Counter
module modulo_n_counter #(
```

```

    parameter N = 10          // Count from 0 to N-1
)(
    input  wire                clk,
    input  wire                rst,
    output reg [$clog2(N)-1:0] count // Auto-sized for N
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 0;
        else begin
            if (count == N-1)        // At max?
                count <= 0;          // Wrap
            else
                count <= count + 1;  // Increment
        end
    end
endmodule

```

Special Verilog Function: `$clog2()`

Line 7: output reg [\$clog2(N)-1:0] count

What is `$clog2()`?

- **Ceiling Log Base 2**
- Calculates minimum bits needed to represent N values

Examples:

```

$clog2(10) = 4    (need 4 bits for 0-9)
$clog2(16) = 4    (need 4 bits for 0-15)
$clog2(100) = 7   (need 7 bits for 0-99)
$clog2(256) = 8   (need 8 bits for 0-255)

```

Why use it?

- Counter width **automatically adjusts** based on N
 - No manual calculation needed
 - Prevents waste (don't use 8 bits when 4 is enough)
-

Design 5: Full-Featured Counter (All Features Combined)

// full_counter.v - Counter with All Features

```

module full_counter #(
    parameter WIDTH = 8
)(
    input  wire                clk,
    input  wire                rst,
    input  wire                en,          // Enable
    input  wire                load,        // Load control
    input  wire                up_down,     // Direction: 1=up, 0=down
    input  wire [WIDTH-1:0]    data,       // Load value
    output reg [WIDTH-1:0]    count
);

```

```

always @(posedge clk or posedge rst) begin
    if (rst)
        count <= {WIDTH{1'b0}};           // Reset
    else if (load)
        count <= data;                     // Load (priority 2)
    else if (en) begin                     // Enabled? (priority 3)
        if (up_down)
            count <= count + 1'b1;         // Count up
        else
            count <= count - 1'b1;         // Count down
    end
    // else: hold (en=0)
end
endmodule

```

Priority: Reset > Load > Enable > Hold

Testbench: Testing All Counter Types

```

// tb_counters.v - Testbench for Advanced Counters
module tb_counters;

    reg clk, rst;

    // Up-Down Counter
    reg up_down;
    wire [7:0] count_updown;

    // Counter with Load
    reg load;
    reg [7:0] data;
    wire [7:0] count_load;

    // BCD Counter
    wire [3:0] count_bcd;

    // Instantiate counters
    up_down_counter uut_updown (
        .clk(clk),
        .rst(rst),
        .up_down(up_down),
        .count(count_updown)
    );

    counter_with_load uut_load (
        .clk(clk),
        .rst(rst),
        .load(load),
        .data(data),
        .count(count_load)
    );

    bcd_counter uut_bcd (
        .clk(clk),
        .rst(rst),
        .count(count_bcd)
    );

    // Clock generator

```

```

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

// Test stimulus
initial begin
    $dumpfile("counters.vcd");
    $dumpvars(0, tb_counters);

    $display("Time | UpDn | Load | BCD");
    $display("-----");
    $monitor("%4t | %3d | %3d | %2d",
        $time, count_updown, count_load, count_bcd);

    // Initialize
    rst = 1; up_down = 1; load = 0; data = 8'd0;
    #10;
    rst = 0;
    #10;

    // Test 1: Count up
    $display("\n--- Test 1: Count Up ---");
    up_down = 1;
    #50;

    // Test 2: Count down
    $display("\n--- Test 2: Count Down ---");
    up_down = 0;
    #50;

    // Test 3: Load value
    $display("\n--- Test 3: Load 100 ---");
    data = 8'd100;
    load = 1;
    #10;
    load = 0;
    #30;

    // Test 4: Watch BCD wrap from 9 to 0
    $display("\n--- Test 4: BCD Wrap (wait for 9→0) ---");
    #100;

    $finish;
end

endmodule

```

Mini Task (8 min)

Build a Divide-by-12 Counter (for a clock)

Requirements:

1. Counts from 0 to 11, then wraps to 0
2. Has enable input (counts only when enabled)
3. Has async reset
4. Output the current count

Use case: 12-hour clock (0-11 for hours)

Template:

```
module div12_counter (
    input wire    clk,
    input wire    rst,
    input wire    en,
    output reg [3:0] count // 4 bits for 0-11
);

    // TODO: Your code here

endmodule
```

Hint: Combine the modulo-N pattern with enable control.

Write the module and a simple testbench. Show me the output where the counter reaches 11 and wraps to 0!

Once verified, **Day 3 is COMPLETE!** 🎉

Then we move to **Day 4: Finite State Machines** (the most important topic in digital design). 🚀